SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Analytical Problem Solving: 40%

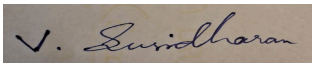
QUESTIONS: 5 **Total Marks: 50**

Annexure A

Code of Conduct:

I V.Susidharan (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Question 1: RSA Key Generation and Security Analysis

Given: p = 17, q = 19./0, e = 5, message M = 7

Step 1: Compute n and φ(n)

$n = p \times q = 17 \times 19 = 323$

$\varphi(n) = (p - 1)(q - 1) = 16 \times 18 = 288$

Step 2: Choose e = 5 and calculate the private key d

e = 5 is valid since gcd(5, 288) = 1 ($288 = 2^5 \times 3^2$, and 5 divides neither factor).

d is the modular inverse of e mod φ(n): find d such that $5d \equiv 1 \pmod{288}$.

Using the Extended Euclidean Algorithm on 288 and 5:

Step	Equation
1	$288 = 57 \times 5 + 3$
2	$5 = 1 \times 3 + 2$
3	$3 = 1 \times 2 + 1$
4	$2 = 2 \times 1 + 0$ (gcd = 1)

Back-substituting gives $5 \times (-115) + 288 \times 2 = 1$, so $d \equiv -115 \pmod{288} \equiv 173 \pmod{288}$.

Check: $5 \times 173 = 865 = 3 \times 288 + 1$ ✓

Private key d = 173

Step 3: Encrypt the message M = 7

$C = M^e \bmod n = 7^5 \bmod 323$

Power of 7	Value mod 323
7^2	49
7^3	343 mod 323 = 20
7^4	20 × 7 = 140
7^5	140 × 7 = 980 mod 323 = 11

Ciphertext C = 11

Step 4: Decrypt the ciphertext and verify correctness

$M = C^d \bmod n = 11^{173} \bmod 323$, computed by fast (square-and-multiply) exponentiation using the binary expansion 173 = 128 + 32 + 8 + 4 + 1:

Term	Value mod 323

11^1	11
11^4	106
11^8	254
11^{32}	273
11^{128}	273
Product $11^{128}\times11^{32}\times11^{8}\times11^{4}\times11^{1}$ mod 323	= 7

Decrypted message M = 7, which matches the original plaintext — encryption/decryption verified correct.

Step 5: Impact of choosing small prime numbers on RSA security

With p = 17 and q = 19, n = 323 is small enough to factor by simple trial division (its square root is under 18), so an attacker can recover p and q almost instantly, then compute φ(n) and derive the private key d exactly as done above. This completely breaks RSA. In real deployments, p and q must each be very large (currently at least 1024–2048-bit primes, i.e., hundreds of decimal digits) so that factoring n is computationally infeasible even with the best known algorithms (e.g., the General Number Field Sieve). This example uses small primes purely to make the arithmetic tractable by hand; it would offer no real security in practice.

Question 2: ElGamal Encryption Analysis

Given: p = 23, g = 5, private key x = 7, random k = 3, message M = 10

Step 1: Compute the public key

$y = g^x \bmod p = 5^7 \bmod 23$

Power of 5	**Value mod 23**
5^1	5
5^2	25 mod 23 = 2
5^3	10
5^4	4
5^5	20
5^6	8
5^7	17

Public key: (p = 23, g = 5, y = 17)

Step 2: Encryption — generate ciphertext pair (C1, C2)

$C1 = g^k \bmod p = 5^3 \bmod 23 = 10$

$C2 = M \times y^k \bmod p = 10 \times 17^3 \bmod 23$

$17^2 \bmod 23 = 289 \bmod 23 = 13$; $17^3 \bmod 23 = 17 \times 13 = 221 \bmod 23 = 14$

$C2 = 10 \times 14 \bmod 23 = 140 \bmod 23 = 2$

Ciphertext (C1, C2) = (10, 2)

Step 3: Decrypt the ciphertext

Compute $C1^x \bmod p = 10^7 \bmod 23 = 14$ (via repeated squaring/multiplication).

Find the modular inverse of 14 mod 23 using the Extended Euclidean Algorithm: $14^{-1} \bmod 23 = 5$ (check: $14 \times 5 = 70 = 3 \times 23 + 1$ ✓).

$M = C2 \times (C1^x)^{-1} \bmod p = 2 \times 5 \bmod 23 = 10$

Decrypted message M = 10, matching the original plaintext — decryption verified correct.

Step 4: How randomness (k) improves security in ElGamal

The random value k makes ElGamal encryption non-deterministic: encrypting the same message M twice with two different values of k produces two completely different ciphertext pairs (C1, C2). This prevents an attacker from recognizing repeated plaintexts by comparing ciphertexts and defeats simple pattern/frequency analysis, giving ElGamal semantic security. However, k must be kept secret and never reused across different encryptions — if the same k is used for two messages, an attacker who learns one plaintext can algebraically recover the other message (and potentially the private key x), since the two C2 values would share the same y^k factor.

Question 3: RSA Digital Signature Verification

Given: p = 11, q = 17, e = 7, message M = 8

Step 1: Compute n and φ(n)

$n = p \times q = 11 \times 17 = 187$

$\varphi(n) = (p - 1)(q - 1) = 10 \times 16 = 160$

Step 2: Find the private key d

d is the modular inverse of e = 7 mod 160: solve $7d \equiv 1 \pmod{160}$.

Step	Equation
1	$160 = 22 \times 7 + 6$
2	$7 = 1 \times 6 + 1$
3	$6 = 6 \times 1 + 0$ (gcd = 1)

Back-substitution: $1 = 7 - 1 \times 6 = 7 - (160 - 22 \times 7) = 23 \times 7 - 160$, so $d \equiv 23 \pmod{160}$.

Check: $7 \times 23 = 161 = 160 + 1$ ✓

Private key d = 23

Step 3: Generate the digital signature for the message

In RSA signing, the sender signs with their own private key: $S = M^d \bmod n = 8^{23} \bmod 187$ (using square-and-multiply, since $23 = 16 + 4 + 2 + 1$):

Term	Value mod 187
8^{16}	69
8^{4}	169
8^{2}	64
8^{1}	8
S = 69×169×64×8 mod 187	= 83

Digital signature S = 83

Step 4: Verify the signature using the public key

Verification recovers the message from the signature using the public key: $M' = S^e \bmod n = 83^7 \bmod 187$.

$83^2 \bmod 187 = 157$; $83^4 \bmod 187 = 152$; since $7 = 4 + 2 + 1$:

$M' = 83^4 \times 83^2 \times 83^1 \bmod 187 = 152 \times 157 \times 83 \bmod 187 = 8$

M' = 8, which matches the original message M — signature verified successfully.

Step 5: How digital signatures ensure non-repudiation

A valid signature S can only be produced using the signer's private key d, which is known exclusively to the signer. Anyone can verify the signature using the signer's public key e, but no one other than the private-key holder could have generated a signature that verifies correctly. Consequently, once a message is signed, the signer cannot later deny having signed it — this is the property of non-repudiation. It also provides integrity: any change to M after signing would cause the verification step to fail, revealing tampering.

Question 4: Diffie-Hellman with Attack Resistance Evaluation

Given: p = 29, g = 2, User A private key a = 5, User B private key b = 12

Step 1: Compute public keys for both users

A's public key: $A = g^a \bmod p = 2^5 \bmod 29 = 32 \bmod 29 = 3$

B's public key: $B = g^b \bmod p = 2^{12} \bmod 29 = 4096 \bmod 29 = 7$

Step 2: Compute the shared secret key

A computes: $K = B^a \bmod p = 7^5 \bmod 29 = 16$

B computes: $K = A^b \bmod p = 3^{12} \bmod 29 = 16$

Both parties independently arrive at the same shared secret key K = 16.

Step 3: What happens if an attacker intercepts and replaces public keys

Basic Diffie-Hellman provides no authentication of the exchanged public keys, so it is vulnerable to a Man-in-the-Middle (MITM) attack. If an attacker intercepts A's and B's public keys and substitutes their own values (Am and Bm), the attacker ends up sharing one secret key with A and a different secret key with B, while both A and B believe they share a single secret directly with each other. The attacker can then transparently decrypt, read,

optionally modify, and re-encrypt every message passing between them, without either party detecting the interception.

Step 4: Cryptographic solution to ensure authenticity

The exchanged public keys must be authenticated before use. Practical solutions include: (1) having each party digitally sign their public key (e.g., using RSA or DSA) so the other party can verify it came from the genuine sender, typically backed by a certificate issued by a trusted Certificate Authority (PKI); (2) using an authenticated key-exchange protocol such as the Station-to-Station (STS) protocol, which combines Diffie-Hellman with digital signatures; or (3) using a password-authenticated key exchange (PAKE) when a shared secret/password already exists between the parties. Any of these prevents an attacker from impersonating either party during the exchange.

Question 5: Hybrid Encryption System Analysis

Given: RSA is used once for key exchange (5 ms), AES encrypts each of 500 data blocks (0.5 ms per block).

Step 1: Calculate total time taken for hybrid encryption

Hybrid time = RSA key-exchange time + (AES time per block × number of blocks)

= 5 ms + (0.5 ms × 500)

= 5 ms + 250 ms = 255 ms

Step 2: Compare with using only RSA for encrypting all data

RSA-only time = RSA time per block × number of blocks = 5 ms × 500 = 2500 ms

Approach	Total time	Relative speed
Hybrid (RSA + AES)	255 ms	≈ 9.8× faster
RSA only (all 500 blocks)	2500 ms	baseline

Step 3: Why hybrid encryption is more efficient

RSA (asymmetric) relies on expensive large-integer modular exponentiation, making it far slower per block than a symmetric cipher. AES (symmetric) is designed for fast bulk data processing using simple byte/word-level operations, making each block roughly 10× cheaper to encrypt in this scenario. By using RSA only once — to securely establish/exchange a symmetric session key — and then using AES for all 500 data blocks, the system pays the expensive asymmetric cost only a single time instead of 500 times, which is why the hybrid total (255 ms) is dramatically lower than the RSA-only total (2500 ms).

Step 4: Justification for hybrid encryption in modern secure communication

Hybrid encryption combines the best properties of both cryptographic families: RSA (or another public-key algorithm) solves the key-distribution problem by allowing two parties to securely agree on a secret session key without having shared one in advance, while AES provides the speed needed to encrypt large volumes of actual data efficiently. This is exactly the model used by real-world secure communication protocols — for example, TLS/SSL (which secures HTTPS) uses RSA or ECDHE during the handshake to establish a session key, and then uses AES to encrypt the actual application data for the rest of the session. This combination delivers both strong security and practical performance at scale, which pure asymmetric or pure symmetric encryption cannot achieve alone.